FILE: C:\HCL_TRAINING\server\src\middleware\auth.js

LINES: 64 | SIZE: 1.7 KB | LANGUAGE: JS

MODULE: JWT HTTP-Only Cookie & Bearer Token Verification with Resilient Fallback

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const protect = async (req, res, next) => {
  let token;

  // 1. Check HTTP-only cookie
  if (req.cookies && req.cookies.token) {
    token = req.cookies.token;
  }
  // 2. Check Authorization header
  else if (req.headers.authorization && req.headers.authorization.startsWith('Bearer'))
> {
    token = req.headers.authorization.split(' ')[1];
  }

  if (!token) {
    return res.status(401).json({
      success: false,
      message: 'Not authorized, no token provided',
    });
  }

  try {
    const secret = process.env.JWT_SECRET || 'hiresense_super_secure_jwt_secret_key_2026
> _recruit_intelligence';
    const decoded = jwt.verify(token, secret);

    let user = await User.findById(decoded.id).select('-password');
    if (!user) {
      // Fallback for dev / in-memory server restarts where ObjectIDs regenerated
      user = await User.findOne({ email: 'recruiter@hiresense.ai' }).select('-password')
> ;
    }

    if (!user) {
      return res.status(401).json({
        success: false,
        message: 'The user belonging to this token no longer exists',
      });
    }

    req.user = user;
    req.organizationId = user.organization;
    next();
  } catch (error) {
    return res.status(401).json({
      success: false,
      message: 'Token verification failed or expired',
    });
  }
};

const authorize = (...roles) => {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        message: `User role '${req.user.role}' is not authorized to access this route`,
      });
    }
    next();
  };
};

module.exports = { protect, authorize };
```

64 |